```
      "must": [
         { "match": {"FIELD": "TEXT"}},
{ "term": {"FIELD": {"value": "VALUE"}}}
      ],
      "must_not": [
        {"bool": { "must": [{}]}}}
      ]
      "should": [
         { "range": { "FIELD": {"gte": 10,"lte": 20}}},
{ "terms": { "FIELD": [ "VALUE1", "VALUE2" ]}}
      ]
    }
  }
}
```

Here, we have three clauses, and each clause holds leaf queries such as `match`, `term`, `range`, and so on. We also have a compound clause in the `must_not` clause, where we can further expand our criteria using the same set of clauses. These individual queries joined in clauses let us write a search query that satisfies advanced query requirements.

While you may understand the previous query from a theoretical perspective, unless you write and execute queries, you can't appreciate their full potential. Let's examine the `bool` query from the ground up, building a query one clause at a time and evolving it as we go. We start our sprint with the `must` clause.

11.3.2 The must clause

The criterion we declare in a `bool` query's `must` clause yields positive results when the queries defined in the (clause) block satisfy the criterion. That is, the output contains all the documents that match the conditions in the `must` clause.

Let's explore a simple query to begin with. Suppose our requirement is to find all TVs in the `products` index. For that, we write a `bool` query with a `must` clause. Because we are looking only for TVs, we can put the search criterion in a `match` query, matching products that are TVs. The following listing provides the code.

Listing 11.2 A `bool` query's `must` clause with a `match` query

```
GET products/_search
{
  "query": {
    "bool": {
      "must": [
        {
          "match": {
            "product": "TV"
          }
        }
      ]
    }
  }
}
```